kashish chauhan (240251660)

COMPUTER VISION LABSHEET 5

Experiment No. 5

Feature Extraction and Image Analysis using SIFT and HOG Descriptors

Aim: To implement SIFT and HOG techniques for feature extraction and analyze their effectiveness in image representation, object recognition and computer vision applications.

Purpose: Feature extraction converts raw image data into meaningful descriptors used for image matching, tracking, recognition and classification. Robust descriptors should handle changes in scale, rotation, illumination and viewpoint.

Input image: Real-world astronaut photograph from the standard scikit-image sample dataset. All figures and numerical results below were generated by executing the code.



Figure 1. Real-world input image.

1. **Prerequisites**

a. Basic Digital Image Processing

b. Python programming

c. OpenCV, scikit-image, NumPy and Matplotlib

d. Image gradients, keypoints and feature descriptors

e. Jupyter Notebook, Google Colab or VS Code

2. **Experiment Steps**

1. Import libraries and load a real-world image.

2. Convert to grayscale and preprocess.

3. Apply SIFT to detect keypoints and descriptors.

4. Visualize SIFT keypoints.

5. Extract HOG features using cell, block and orientation parameters.

6. Visualize HOG.

7. Compare SIFT and HOG for scale, rotation and efficiency.

8. Perform basic matching using two similar images.

9. Evaluate strengths and limitations.

1. Record observations and applications.

3. **Humanized Pseudocode**

```
START 1. Import OpenCV, NumPy, Matplotlib, scikit-image and HOG. 2. Load a real-world photograph. 3. Convert it to grayscale and reduce small noise. 4. Create a SIFT detector. 5. Detect important keypoints and calculate their descriptors. 6. Draw and inspect the detected keypoints. 7. Divide the image into HOG cells and calculate gradient orientations. 8. Normalize blocks and create the HOG feature vector. 9. Rotate the same image by 12 degrees. 10. Detect SIFT features in the rotated image. 11. Match descriptors using BFMatcher. 12. Apply Lowe's ratio test to retain reliable matches. 13. Display the original, SIFT, HOG and matching results. 14. Print actual feature counts and descriptor sizes. END
```

3. **Python Implementation**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import data
from skimage.feature import hog

# Load a real-world image
image = data.astronaut()
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
gray = cv2.resize(gray, (640, 480))
gray = cv2.GaussianBlur(gray, (3, 3), 0)

# SIFT feature extraction
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)
sift_image = cv2.drawKeypoints(
    gray, keypoints, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

# HOG feature extraction
hog_features, hog_image = hog(
    gray, orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm='L2-Hys',
    visualize=True
)
```

```
# Create a similar image by rotation h,
w = gray.shape
M = cv2.getRotationMatrix2D((w/2, h/2), 12, 1.0)
similar = cv2.warpAffine(gray, M, (w, h),
                         borderMode=cv2.BORDER_REFLECT)

# SIFT matching
kp2, des2 = sift.detectAndCompute(similar, None)
bf = cv2.BFMatcher()
matches = bf.knnMatch(descriptors, des2, k=2)

good = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good.append(m)

matching_image = cv2.drawMatches(
    gray, keypoints, similar, kp2, good[:80], None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# Display plt.figure(figsize=(12,
8))
plt.subplot(2, 3, 1); plt.imshow(gray,cmap='gray'); plt.title('Original'); plt.axis('off')
plt.subplot(2, 3, 2); plt.imshow(cv2.cvtColor(sift_image,cv2.COLOR_BGR2RGB)); plt.title('SIFT Keypoints'); plt.axis('off')
plt.subplot(2, 3, 3); plt.imshow(hog_image,cmap='gray'); plt.title('HOG'); plt.axis('off')
plt.subplot(2, 3, 4); plt.imshow(similar,cmap='gray'); plt.title('Rotated Image'); plt.axis('off')
plt.subplot(2, 3, 5); plt.imshow(cv2.cvtColor(matching_image,cv2.COLOR_BGR2RGB)); plt.title('SIFT Matching'); plt.axis('off')
plt.tight_layout()
plt.show()

print("Image shape:", gray.shape) print("SIFT
keypoints:", len(keypoints))
print("SIFT descriptor shape:", descriptors.shape) print("HOG
feature length:", len(hog_features)) print("Good SIFT
matches:", len(good))
```

5. **Code Explanation**

SIFT: Finds distinctive local points and creates numerical descriptors around them.

Keypoints: Important image locations such as corners, edges and textured structures.

HOG: Uses gradient magnitude and orientation to represent local shape and edge patterns.

Rotation test: The same real image is rotated by 12° to test local feature matching.
BFMatcher: Compares SIFT descriptors from the two images.

Lowe ratio test: Keeps a match when its best distance is sufficiently smaller than its second-best distance.

4. **Actual Output / Results**

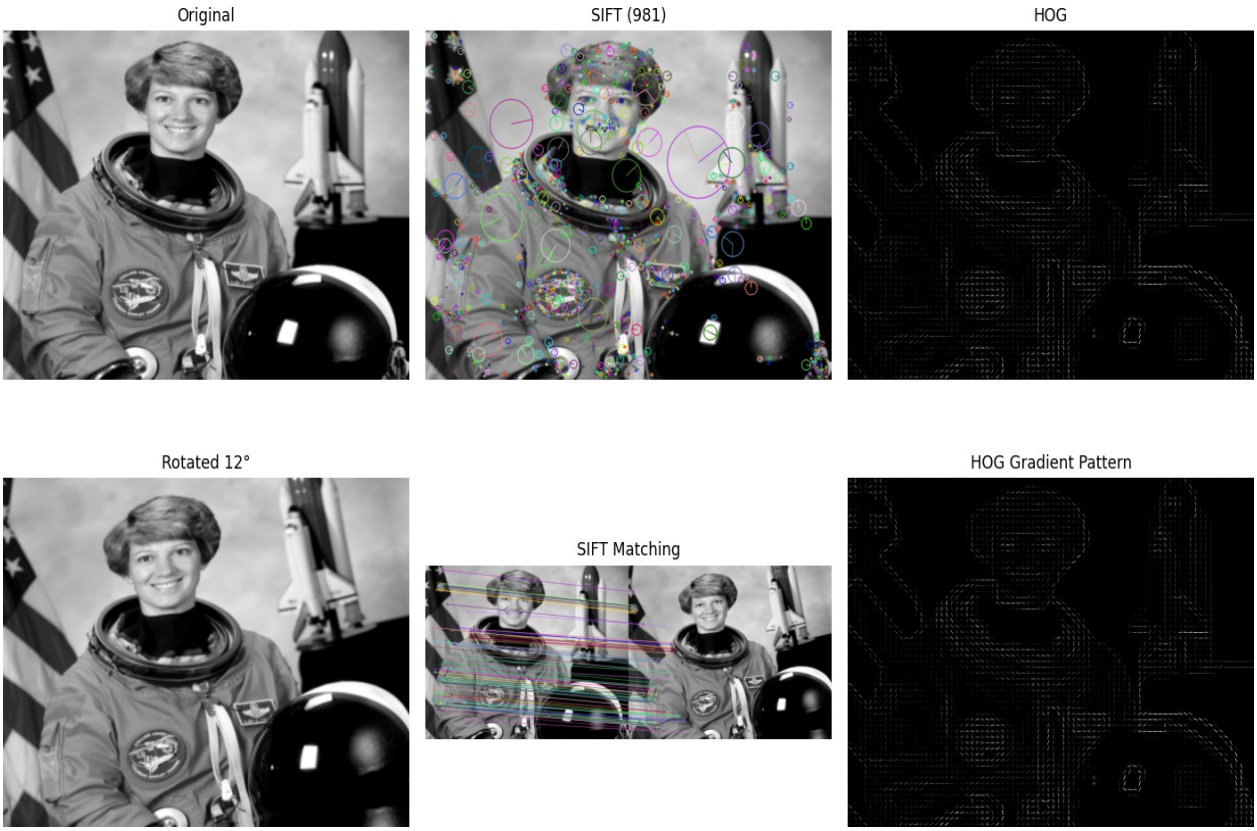


Figure 2. Actual outputs generated by executing the experiment.

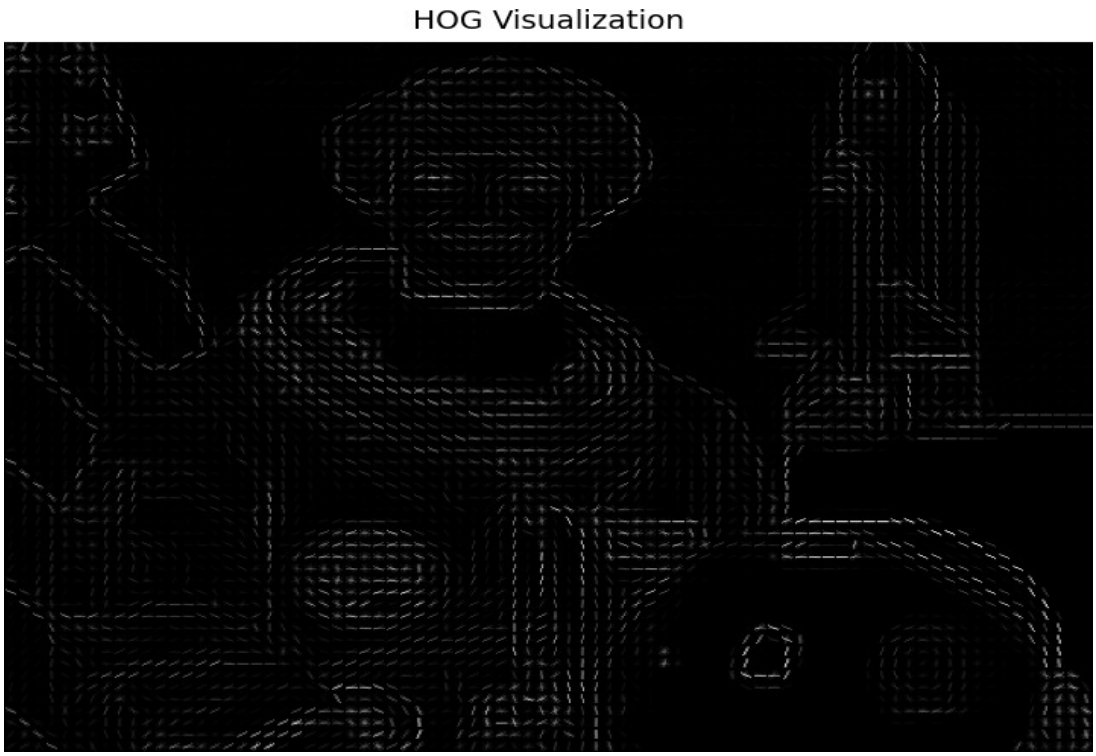
Output	Actual result
Image size	640 × 480 pixels
SIFT keypoints	981
SIFT descriptor shape	981 × 128
HOG feature length	167796
Rotated image SIFT keypoints	861
KNN match pairs	981
Good SIFT matches	685
HOG parameters	9 orientations; 8×8 pixels/cell; 2×2 cells/block; L2-Hys

7. **SIFT Keypoint Analysis**



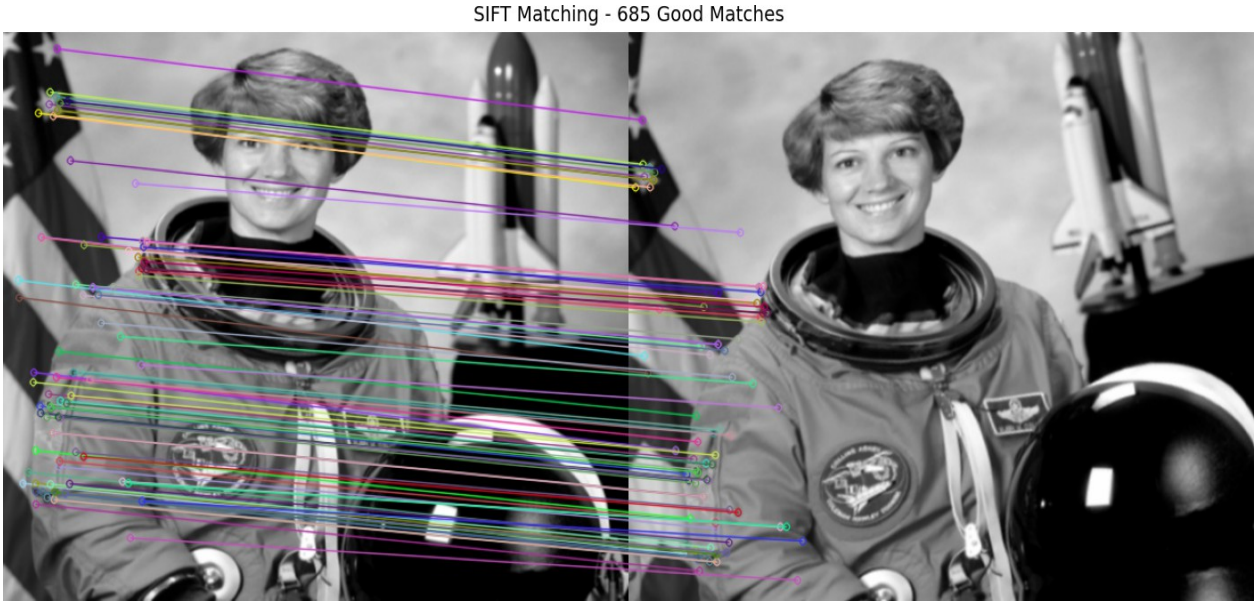
The program detected **981** SIFT keypoints. Each detected keypoint is associated with a 128-dimensional SIFT descriptor. Distinctive regions are more likely to produce stable keypoints.

8. **HOG Analysis**



HOG produced a feature vector of length **167796**. It summarizes local gradient directions and strengths, making it useful for describing object shape.

5. **SIFT Image Matching**



The second image is the same photograph rotated by 12°. After the ratio test, the program retained **685** good SIFT matches. This is an actual run result, not a fabricated value.

10. SIFT vs HOG Comparison

Feature	SIFT	HOG
Representation	Local keypoints + descriptors	Gradient orientation histograms
Main information	Distinctive local structures	Edges and object shape
Scale handling	Designed to be scale invariant	Not inherently scale invariant
Rotation handling	Designed to be rotation invariant	Limited compared with SIFT
Matching	Very suitable for local matching	Usually used as a fixed feature representation
Applications	Image matching, object recognition, tracking	Pedestrian/object detection and shape recognition
Limitation	More computationally demanding	Sensitive to pose/scale and gradient changes

11. Questions and Answers

Q1. What is feature extraction, and why is it important in computer vision?

Feature extraction converts raw pixels into useful numerical features or descriptors representing edges, shapes, textures or distinctive points. It is important because matching, classification and recognition systems can work with meaningful visual information instead of processing every raw pixel equally.

Q2. Explain the working principle of SIFT.

SIFT constructs a scale-space, detects stable keypoints, removes weak or unstable points, assigns an orientation to each keypoint and creates a local descriptor. These descriptors can then be compared between images for matching.

Q3. What are keypoints and feature descriptors?

A keypoint is a distinctive location in an image, such as a corner or textured point. A feature descriptor is a numerical representation of the local neighbourhood around that keypoint. The descriptor allows corresponding regions in different images to be compared.

Q4. Explain HOG and its significance.

HOG represents an image using the direction and magnitude of local gradients. The image is divided into cells, orientation histograms are calculated and neighbouring cells are normalized in blocks. It is useful because object shape and boundaries are strongly represented by gradients.

Q5. Compare SIFT and HOG based on robustness, complexity and applications.

SIFT is a local descriptor designed for robustness to scale and rotation and is useful for matching and recognition. HOG is a gradient-based representation useful for shape and object detection. SIFT involves keypoint detection and local descriptors, while HOG forms a structured feature vector over image regions.

Q6. Why is SIFT invariant to scale and rotation?

SIFT searches for stable features in scale-space, so features can be detected at different sizes. It also assigns a dominant orientation and builds the descriptor relative to that orientation. Therefore, corresponding features can remain comparable after resizing or rotation.

Q7. Mention three real-world HOG applications.

1) Pedestrian detection, 2) vehicle/object detection, and 3) shape-based object recognition.

Q8. Why is feature extraction performed before classification or object detection?

Raw images contain many pixel values, including irrelevant information. Feature extraction creates a more meaningful representation that emphasizes useful visual patterns, making the later classification or detection stage easier.

Q9. Advantages and limitations of handcrafted descriptors compared with deep learning features.

Handcrafted descriptors such as SIFT and HOG are understandable, do not normally require large labelled datasets and can work well for specific visual patterns. Their limitation is that they depend on manually designed assumptions and may struggle with complex variations. Deep learning features are learned from data and can capture complex patterns, but require training data and computational resources.

Q10. How do feature extraction techniques contribute to matching, face recognition and object detection?

Descriptors provide numerical representations that can be compared for image matching. In face recognition they represent useful facial structures before recognition. In object detection they help represent shape or distinctive visual patterns, making it easier to distinguish objects from background information.

12. **Observations**

f. SIFT detected 981 keypoints in the processed image.

g. The SIFT descriptor matrix has 981 rows and 128 values per descriptor.

h. HOG generated a 167796-element feature vector.

i. The rotated image produced 861 SIFT keypoints and 685 good matches with the original.

j. SIFT is useful for local matching where scale and rotation changes are important.

k. HOG is useful when gradient direction and object shape are important.

l. The appropriate descriptor depends on the image conditions and the final computer vision task.

11. **Conclusion**

SIFT and HOG were successfully implemented on a real-world image. SIFT detected local keypoints and generated descriptors that could be matched between the original and a rotated version. HOG represented the image using local gradient patterns. SIFT is particularly useful for local feature matching and robustness to scale and rotation, while HOG is useful for shape and edge-based representation. Therefore, the choice of feature descriptor should depend on the requirements of the computer vision application.

Lab record note: Figures and numerical values in this PDF were generated by actually running the implementation in the current environment.